Assignment 1

Student Name: Sameep Das **UID:** 23BCS11375
Branch: BE-CSE **Section/Group:** KRG-1A
Semester: 6th **Date of Performance:** 04/01/26
Subject Name: System Design **Subject Code:** 23CSH-314

Q1. Explain SRP and OCP in detail with proper examples.

1. Single Responsibility Principle (SRP)

Definition

- **SRP states that a class should have only one reason to change**
- In simple words: *A class should do only one job and do it well*

Key Ideas of SRP

- One class = **one responsibility**
- A responsibility is **a reason to change**
- Mixing multiple responsibilities:
  - Makes code harder to understand
  - Increases coupling
  - Makes testing difficult

Why SRP is Important

- Improves **readability**
- Improves **maintainability**
- Makes code **easier to modify and extend**
- Reduces chances of bugs when requirements change

Example (Violation of SRP)

```
class Invoice {
    void calculateTotal() {
        // calculation logic
    }
    void printInvoice() {
        // printing logic
    }
    void saveToDatabase() {
        // database logic
    }
}
```

Problems

- This class has **three responsibilities**:
    1. Business logic (calculation)
    2. Presentation logic (printing)
    3. Persistence logic (database)
- Changes in printing or database logic force changes in Invoice

Correct SRP Implementation

```
class Invoice {
    void calculateTotal() {
        // calculation logic
    }
}
class InvoicePrinter {
    void print(Invoice invoice) {
        // printing logic
    }
}
class InvoiceRepository {
    void save(Invoice invoice) {
        // database logic
    }
}
```

Benefits

- Each class has **one responsibility**
- Changes are **isolated**
- Code is easier to test and extend

2. Open/Closed Principle (OCP)

Definition

- **Software entities should be open for extension but closed for modification**
- Meaning: You should **add new functionality without changing existing code**

Key Ideas of OCP

- Existing code should not be modified
- New behavior should be added using:
  - Inheritance
  - Interfaces
  - Polymorphism

Why OCP is Important

- Prevents breaking existing functionality
- Makes systems more stable
- Encourages scalable design

Example (Violation of OCP)

```
class PaymentProcessor {
    void process(String type) {
        if (type.equals("CARD")) {
            // card payment logic
        } else if (type.equals("UPI")) {
            // upi payment logic
        }
    }
}
```

Problems

- Adding a new payment method requires **modifying this class**
- Violates OCP

Correct OCP Implementation

```
interface PaymentMethod {
    void pay();
}
class CardPayment implements PaymentMethod {
    public void pay() {
        // card logic
    }
}
class UpiPayment implements PaymentMethod {
    public void pay() {
        // upi logic
    }
}
class PaymentProcessor {
    void process(PaymentMethod payment) {
        payment.pay();
    }
}
```

Benefits

- New payment methods can be added without changing PaymentProcessor
- System becomes extensible and stable

Q2. Discuss violations in SRP and OCP along with their fixes

1. SRP Violations and Fixes - Common SRP Violations

God Class

- One class handles:
  - Business logic
  - Database
  - UI
  - Validation

Multiple Reasons to Change

- Changing UI affects business logic
- Changing database affects calculations

Example of SRP Violation

```
class UserManager {
    void addUser() {
        // logic
    }
    void validateUser() {
        // validation
    }
    void saveToDB() {
        // database
    }
    void sendEmail() {
        // email logic
    }
```

```
}
```

Fix for SRP Violations

```
class UserService {
    void addUser() { }
}

class UserValidator {
    void validate() { }
}

class UserRepository {
    void save() { }
}

class EmailService {
    void sendEmail() { }
}
```

Fix Summary

- Split responsibilities into **separate classes**
- Each class has **one clear purpose**

2. OCP Violations and Fixes - Common OCP Violations

1. Excessive if-else or switch cases

- Logic depends on type checking
- New feature requires modifying old code

2. Hardcoded behavior

- Behavior directly embedded inside classes

Example of OCP Violation

```
class DiscountCalculator {
    double calculate(String customerType) {
        if (customerType.equals("REGULAR")) {
            return 5;
        } else if (customerType.equals("PREMIUM")) {
            return 10;
        }
        return 0;
    }
}
```

Fix Using OCP

```
interface DiscountStrategy {
    double getDiscount();
}
class RegularCustomer implements DiscountStrategy {
    public double getDiscount() {
        return 5;
    }
}
class PremiumCustomer implements DiscountStrategy {
    public double getDiscount() {
        return 10;
    }
```

```
}
```

Fix Summary

- Use **abstraction**
- Use **polymorphism**
- Avoid modifying existing logic

Key Differences Between SRP and OCP

Aspect	SRP	OCP
Focus	Responsibility	Extensibility
Goal	One reason to change	Add features safely
Violation Type	Too many duties	Too many condition checks
Fix	Split classes	Use interfaces & inheritance

Q3. High-Level Design (HLD) for an Online Examination System using SRP & OCP

1. System Overview

An **Online Examination System** allows:

- Students to take exams online
- Admins to create and manage exams
- Evaluators to evaluate results
- System to handle authentication, timing, submission, and result generation

2. Major Functional Requirements

- User Authentication (Student / Admin / Evaluator)
- Exam Creation & Management
- Question Management
- Exam Scheduling
- Exam Attempt & Submission
- Result Evaluation
- Report Generation

3. High-Level Architecture Main Layers

1. **Presentation Layer**
2. **Service (Business Logic) Layer**
3. **Domain Layer**